

TDD — Technical Design Document

TDD — Technical Design Document

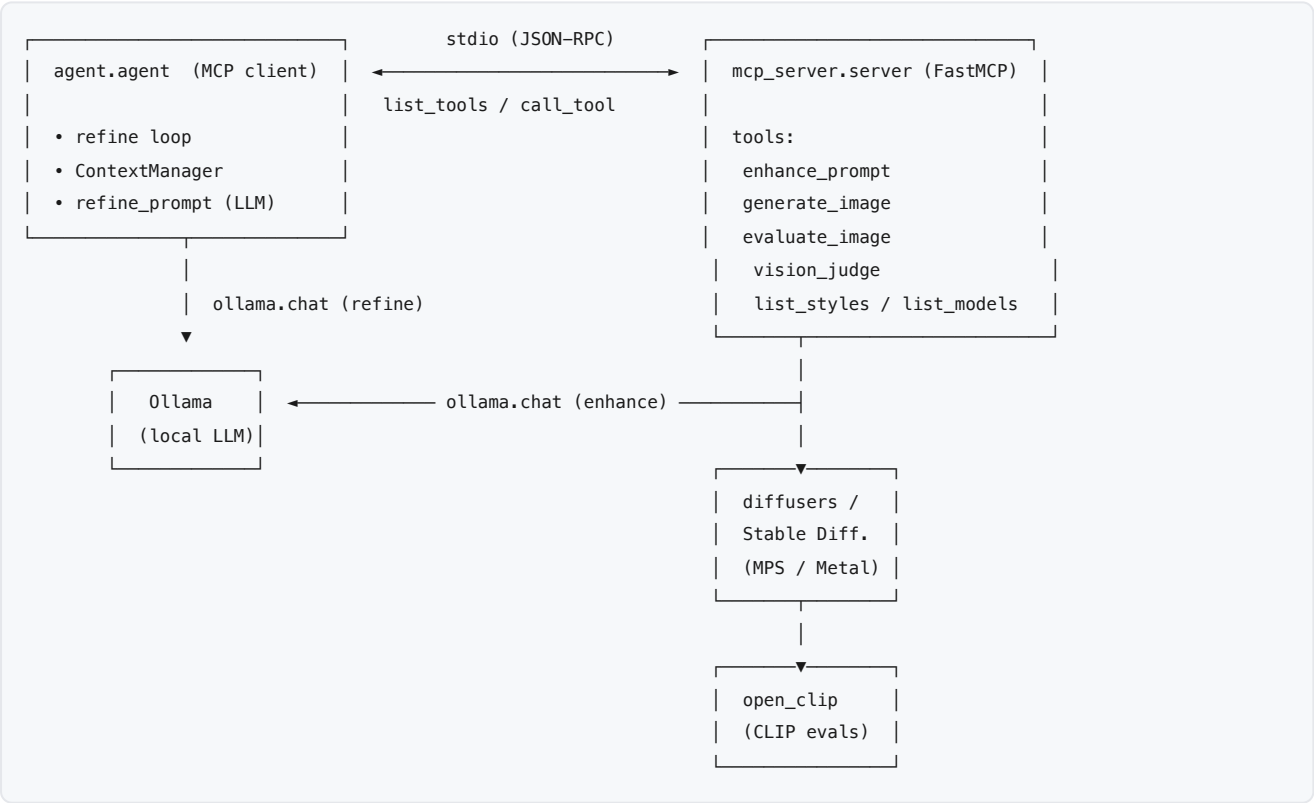
How the semi-3D-anime image-generation agent works, end to end. This is the deep version of the README: it traces every hop from the user's prompt, through the MCP boundary, into the LLM and the diffusion model, the scoring math, and back.

1. The two processes

The system is deliberately split into **two OS processes** that talk over the Model Context Protocol (MCP):

Process	Role	Entry point
Agent harness	MCP client + orchestrator. Owns the refine loop, the context manager, and the refinement LLM call.	<code>agent/agent.py</code>
MCP server	MCP server . A <code>FastMCP</code> instance exposing the image toolset. Owns Stable Diffusion, the enhance LLM call, and the evals.	<code>mcp_server/server.py</code>

They are **not** in the same Python interpreter. The client launches the server as a **subprocess** and speaks to it over **stdio** (newline-delimited JSON-RPC). This is what makes the toolset reusable: the exact same server can be driven by Claude Desktop, an MCP inspector, or any other MCP client — the agent harness is just one such client.



Note that **both** processes import the same `mcp_server/config.py` (`CONFIG`), so they share one configuration source even though they run separately.

2. Transport: how MCP actually carries a call

The agent sets up the server as a subprocess via `StdioServerParameters` (`agent/agent.py:47`):

```
server = StdioServerParameters(
    command=sys.executable,          # the venv's python
    args=["-m", "mcp_server.server"], # launches FastMCP over stdio
    cwd=str(ROOT),
)
async with stdio_client(server) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()    # MCP handshake
        tools = (await session.list_tools()).tools
        result = await session.call_tool("enhance_prompt", {...})
```

What happens on the wire:

1. **Spawn** — `stdio_client` forks `python -m mcp_server.server`. The server's `mcp.run()` (`server.py:84`) starts a FastMCP stdio loop reading JSON-RPC from stdin, writing to stdout.
2. **initialize** — capability/version handshake (MCP protocol).
3. **list_tools** — the server returns the schema of every `@mcp.tool()`-decorated function. FastMCP derives the JSON schema from each function's **Python type hints and docstring**, so the signatures in `server.py` are the public API contract.
4. **call_tool(name, args)** — the client sends a JSON-RPC request; FastMCP deserializes the args, calls the Python function, and serializes the return value into a `CallToolResult`.

Unwrapping the result — `_parse`

A `CallToolResult` is not a plain dict. FastMCP can return the payload two ways, so the agent normalizes both (`agent/agent.py:28`):

- **structuredContent** — for structured returns. FastMCP wraps a *non-dict* return under a `"result"` key, but a dict passes through as-is, so `_parse` does `sc.get("result", sc)`.
- **content blocks** — falls back to the first `text` block and `json.loads` it (or returns `{"text": ...}` if it isn't JSON).

Every tool in this project returns a dict, so in practice the structured path is taken.

3. The orchestration loop, step by step

This is the heart of the system (`agent/agent.py:run`). One full pass:

Step 0 — Bootstrap

```
ctx = ContextManager(system_prompt=SYSTEM_PROMPT, max_tokens=3000)
ctx.add("user", f"Request: {user_request}")
```

`SYSTEM_PROMPT` is read from `agent/system_prompt.txt` (the orchestrator role + style rubric). The `ContextManager` (see §6) keeps the running transcript bounded.

Step 1 — Enhance (MCP → server LLM → back)

```
enh = _parse(await session.call_tool(
    "enhance_prompt", {"user_request": user_request, "style": "semi-3d-anime"}))
subject = enh.get("subject", user_request)
prompt = enh["prompt"]
negative = enh["negative_prompt"]
ctx.add("assistant", f"Enhanced prompt: {prompt}")
```

The call crosses the MCP boundary into `mcp_server/llm_prompt.py::enhance`, which:

1. Calls **Ollama** with a "Stable Diffusion prompt engineer" system prompt that expands the short request into a vivid comma-separated prompt — and explicitly **forbids style keywords** like `3d / anime` (those are added separately, see §4).
2. If Ollama is unreachable, it **falls back to using the raw request** (generation never hard-fails on the LLM).
3. Calls `styles.apply_style()` to append the `semi-3d-anime` positive tags and supply the default negative prompt.
4. Returns `{prompt, negative_prompt, style, subject, style_description}`.

So `subject` = the bare LLM-expanded subject (no style tags); `prompt` = subject + style tags. This separation is what lets the loop refine and re-style cleanly.

Step 2 — Generate (MCP → diffusers → PNG)

```
for i in range(1, max_iters + 1):
    gen = _parse(await session.call_tool("generate_image", {
        "prompt": prompt, "negative_prompt": negative, "style": "semi-3d-anime"}))
    ctx.add("tool", json.dumps(gen), kind="tool_result")
```

This crosses into `sd_pipeline.generate` (see §5). It renders one image on MPS and writes `outputs/img_<timestamp>_<seed>.png` plus a sidecar `.json` of all parameters. The returned dict includes `path`, the final `prompt`, `seed`, `elapsed_sec`, etc.

Step 3 — Evaluate (MCP → CLIP → scores)

```
ev = _parse(await session.call_tool("evaluate_image", {
    "image_path": gen["path"], "prompt": user_request}))
ctx.add("tool", json.dumps(ev), kind="tool_result")
```

Crosses into `evals.evaluate` (see §7). **Important:** the CLIP score is computed against the **original `user_request`**, not the enhanced prompt — we measure whether the image satisfies what the user actually asked for, not the elaborated version of it.

Step 4 — Track best & decide

```
record = {"iteration": i, "image": gen["path"], "prompt": prompt, "eval": ev}
if best is None or ev["composite"] > best["eval"]["composite"]:
    best = record

if ev["passed"]:
    break                    # thresholds met — done
if i == max_iters:
    break                    # give up, keep best so far
```

- **passed** is the *gate* (CLIP $\geq$ threshold **and** style $\geq$ threshold) — it decides when to stop early.
- **composite** is the *ranking key* — it decides which image is returned as best. These are different on purpose (§7).

Step 5 — Refine (client LLM → re-enhance)

If not passed and iterations remain:

```
subject = llm.refine_prompt(subject, prompt, ev)
reenh = _parse(await session.call_tool(
    "enhance_prompt", {"user_request": subject, "style": "semi-3d-anime"}))
prompt, negative = reenh["prompt"], reenh["negative_prompt"]
ctx.add("assistant", f"Refined subject -> {subject}")
```

This is the **second** LLM call site (`agent/llm.py::refine_prompt`, client-side). It inspects the eval result, identifies the **weakest dimension**, and asks the LLM to rewrite *only the subject* (no style keywords):

- If `clip_score < clip_threshold` → "make the subject, action and setting more explicit and prominent."

- If `style_sim < style_threshold` → "describe a glossy stylized 3D character render with big expressive eyes and soft studio lighting."

The rewritten subject is then **fed back through** `enhance_prompt` so style tags are re-applied consistently, and the loop returns to Step 2.

Step 6 — Return

After the loop (and printing `ctx.stats()`), `run` returns `{"request", "best"}`, and `main` prints the final image path and its clip/style/aesthetic scores.

4. Why style tags are applied *outside* the LLM

A recurring design choice: the LLM is **never** asked to produce style keywords. Both LLM system prompts (`llm_prompt.py:_SYS` and `llm.py::refine_prompt`) explicitly say *no 3d / anime* keywords. Instead, `styles.apply_style(subject, style)` (`mcp_server/styles.py`) deterministically appends the preset:

```
positive = f"{subject.strip()}, {preset['positive']}" # subject + style tags
return positive, preset["negative"]                # style supplies negatives
```

The `semi-3d-anime` preset's positive tags lock in "stylized 3D character render, glossy smooth shading, subsurface scattering skin, big expressive eyes ... Pixar-meets-anime ...", and its negative prompt pushes away *flat 2d, line art, photorealistic, ...*. Because the style is applied as a separate, deterministic step, the loop can rewrite the *subject* freely across iterations without the LLM drifting the *style*.

5. Generation: `sd_pipeline.generate`

Backed by `diffusers` (not Ollama — Ollama runs the LLM only and cannot run Stable Diffusion). Key mechanics (`mcp_server/sd_pipeline.py`):

- **Pipeline singleton cache** — `get_pipeline()` lazily loads and caches one pipeline per model name in `_PIPELINES`, so repeated generations don't re-load the ~2GB model.
- **Model registry** — `config.yaml` `image.models` maps a name → {repo, arch}. `arch: sd15` → `StableDiffusionPipeline`; `arch: sd-xl` → `StableDiffusionXLPipeline`. Default is `dreamshaper-8` (SD1.5, HuggingFace-hosted, loaded via `from_pretrained`).
- **Device resolution** — `mps` if available, else `cuda`, else `cpu`.
- **dtype gotcha** — config uses `float32` on MPS deliberately: fp16 SD1.5 VAE produces NaNs/black images on Metal. CPU also forces fp32. (The module docstring still says "fp16" — the config value wins.)
- **Memory** — `enable_attention_slicing()` keeps it friendly on unified-memory Macs.
- **Determinism** — `seed` of `-1 / None` becomes a time-derived random seed; the generator is CPU-pinned (`torch.Generator(device="cpu")`) for reproducibility across backends.
- **Output** — saves the PNG and a sidecar `.json` with every parameter (prompt, negative, model, steps, guidance, dims, seed, device, elapsed). Returns that metadata dict.

Defaults (config): `steps=25`, `guidance_scale=7.0`, `768×512`, ~20–70 s/image on an M1 Pro.

6. Context management

The agent keeps a bounded transcript so a long refine loop doesn't blow up context (`agent/context.py`). It's a custom, dependency-free manager (no tokenizer):

- **Token proxy** — `_approx_tokens(text) ≈ words × 1.3`. Cheap, no tokenizer dependency.
- **add(role, content, kind)** — appends a turn and immediately `compact()` s. Bulky tool output is tagged `kind="tool_result"`.

- `compact()` runs a two-stage shrink when over `max_tokens` (3000):
 1. **Collapse** — truncate older (`tool_result`) turns beyond the most recent `keep_recent` (6) to ~160 chars + `...[tool result truncated]`.
 2. **Fold** — if still over budget, fold all-but-recent turns into a single rolling `_summary` string (and cap the summary itself at `max_tokens // 2`).
- `render()` — produces the message list (`system` prompt, optional summary as a second system message, then turns) for an LLM call.
- `stats()` — `{turns, approx_tokens, max_tokens, summarized}` , printed at the end of a run.

In the current loop, the manager primarily protects against accumulating image-metadata and eval blobs; `render()` exists for sending the full transcript to an LLM if the orchestrator is extended to reason over it.

7. Evaluation: how images are scored

All scoring is CLIP-based (`mcp_server/evals.py`), using `open_clip ViT-B/32` (`laion2b_s34b_b79k`), loaded once and reused (`_load_clip` is `lru_cache` d). The model runs on MPS if available.

Metric	Function	What it measures	Range
<code>clip_score</code>	<code>clip_score(image, prompt)</code>	Cosine similarity between the image embedding and the text prompt embedding — does the image match the request?	~0..0.35 in practice
<code>style_sim</code>	<code>style_sim(image)</code>	Mean cosine similarity between the image embedding and the embeddings of every reference sample in <code>images/</code> — does it look like the target style?	0..1
<code>aesthetic</code>	<code>aesthetic(image)</code>	A proxy mapped from the CLIP image-feature L2 norm onto ~3..9. Not a trained aesthetic predictor — avoids an extra model download; relative signal only.	~0..10

All embeddings are L2-normalized before the dot product, so cosine similarity is just `a @ b.T`. `style_sim` averages over all reference samples (the embeddings of `images/*` are cached in `_sample_features` , also `lru_cache` d).

The verdict — `evaluate(image_path, prompt)`

```
cs = clip_score(image_path, prompt)
ss = style_sim(image_path)
aes = aesthetic(image_path)
passed = cs >= clip_threshold and ss >= style_threshold
composite = round(cs + ss + aes / 20.0, 4)
```

- **passed** — the early-stop gate. Both thresholds must be met. In-loop thresholds come from `config.yaml` (`clip_threshold: 0.21` , `style_threshold: 0.50`). These are calibrated to ViT-B/32 raw cosine, where image-text alignment rarely exceeds ~0.30 even for strong matches — so 0.21 is a realistic bar for stylized art.
- **composite** — the cross-iteration ranking key. `aesthetic` is divided by 20 to bring its ~0..10 range into rough parity with the two cosine terms.

Optional vision judge

`vision_judge(image, prompt)` sends the image to a **multimodal Ollama model** (`llm.vision_model` , default `gemma4:12b`) for a 2–3 sentence qualitative grade + score/10. It is **off the loop path** (`evals.enable_vision_judge: false`) because it's slow; it's exposed as its own MCP tool for ad-hoc use.

Batch evals (regression check)

`evals/run_evals.py` is a separate harness from the in-loop scoring. It generates one image per prompt in `evals/eval_dataset.json` , scores each, and prints a pass/fail table. It uses the **stricter thresholds defined in the dataset** (`clip_score: 0.25` , `style_sim: 0.55`), not the in-loop config thresholds — so it's a deliberately tougher regression bar.

8. Which LLM, and where it's called

There is **one LLM provider** — **local Ollama** — invoked at three distinct sites:

Site	File	Purpose	Model used
enhance	mcp_server/llm_prompt.py (server-side, inside enhance_prompt tool)	Expand short request → rich SD subject prompt	config.llm.model (default qwen3.5:4b)
refine	agent/llm.py::refine_prompt (client-side, in the loop)	Rewrite the subject to fix the weakest eval dimension	config.llm.model
vision judge	mcp_server/evals.py::vision_judge (server-side, optional)	Qualitative multimodal grade	config.llm.vision_model (default gemma4:12b)

All text calls go through `ollama.chat(...)` with `options.temperature` from config (`llm.temperature: 0.7`). The agent's `agent/llm.py::chat` is a thin wrapper used by `refine_prompt` . Crucially, **every LLM call degrades gracefully**: if Ollama is unavailable, `enhance` falls back to the raw request and `refine_prompt` returns the previous subject, so a missing LLM never breaks image generation — it just removes the smarts.

9. End-to-end trace (one request, two iterations)

```
user: "two friends playing video games on a couch"
|
├─ ctx.add(user)                                [client]
├─ spawn `python -m mcp_server.server` over stdio [client→server]
├─ session.initialize / list_tools                [MCP handshake]
|
├─ call_tool enhance_prompt(request, style)        [MCP → server]
|   └─ ollama.chat (qwen3.5:4b) → subject prompt  [server → Ollama]
|   └─ apply_style → +positive tags, +negatives    [server]
|   └─ {prompt, negative, subject}                 [server → MCP → client]
|
├─ iter 1: call_tool generate_image(prompt, negative) [MCP → server]
|   └─ get_pipeline (cached) → diffusers on MPS → PNG [server]
|   └─ {path, seed, elapsed, ...}
|   └─ call_tool evaluate_image(path, ORIGINAL request) [MCP → server]
|       └─ open_clip ViT-B/32: clip, style_sim, aesthetic [server]
|       └─ {clip_score, style_sim, aesthetic, composite, passed}
|   └─ passed? NO → best = iter1
|       └─ refine_prompt(subject, prompt, eval)        [client → Ollama]
|       └─ re-enhance_prompt(new subject)              [MCP → server → Ollama]
|
├─ iter 2: generate_image → evaluate_image          [MCP → server]
|   └─ passed? YES → best = (whichever composite is higher)
|
└─ print FINAL RESULT: best image path + scores    [client]
```

10. File map (for navigation)

File	Responsibility
agent/agent.py	MCP client; the generate→evaluate→refine loop; <code>_parse</code> for tool results
agent/llm.py	Ollama wrapper + <code>refine_prompt</code> (client-side LLM call)
agent/context.py	<code>ContextManager</code> (bounded transcript, summarization)
agent/system_prompt.txt	Orchestrator role + style rubric
mcp_server/server.py	FastMCP server; tool definitions (the public API surface)
mcp_server/llm_prompt.py	<code>enhance</code> (server-side LLM call) + graceful fallback
mcp_server/sd_pipeline.py	diffusers wrapper; pipeline cache; PNG + metadata output
mcp_server/evals.py	CLIP scoring (clip/style/aesthetic), verdict, vision judge
mcp_server/styles.py	Style presets; <code>apply_style</code> (deterministic tag injection)
mcp_server/config.py	Loads <code>config.yaml</code> once into the shared <code>CONFIG</code> singleton
evals/run_evals.py	Batch regression harness over <code>eval_dataset.json</code>
config.yaml	Single source of truth: models, device, dtype, thresholds, loop iters